

0. Создайте приложение **app0**

0.1 Выполните команду

```
django-admin.exe startproject app0
```

1. Вносим изменения в файл **settings.py**

1.1 Инициализация приложения

Application definition

```
INSTALLED_APPS = [  
    'app0',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

Добавить название Вашего приложения.
В нашем примере оно называется **app0**

1. Вносим изменения в файл **settings.py**

1.2 Инициализация директории для сайта

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / "templates"],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

Добавить строку. В директории **templates** будут располагаться файлы Вашего сайта (HTML, CSS и пр.)

2. Вносим изменения в файл `urls.py`

2.1 Добавить строки: определяем маршруты для главной страницы (`main.html`) и страницы с каталогом (`catalog.html`)

```
from django.contrib import admin
```

```
from django.urls import path
```

```
from . import views
```

2.2 Добавить строку: подключение конт роллеров

Имя файла, в котором определены конт роллеры

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

Имя маршрута

```
    path('catalog/', views.catalog, name="catalog"),
```

Маршрут

Имя контроллера

```
    path('', views.main, name="main")
```

```
]
```

3. Создайте файл **views.py** (в рамках Вашего проекта)

В этом файле будут определены контроллеры *main* и *catalog*.

4. Вносим изменения в файл **views.py**

4.1 Добавить строку

```
from django.template.response import TemplateResponse
```



Подключение класса для сборки HTML-страницы

4. Вносим изменения в файл **views.py**

4.2 Добавить контроллер **main**

Контроллер, **имя** которого определено в файле **urls.py**

def **main**(request):

Объект, в котором хранятся параметры запроса

-----> return TemplateResponse(request, "**main.html**")

Обязательный отступ клавишей Tab

HTML-файл, который должен отобразиться в веб-браузере пользователя, когда он выполнит переход на главную страницу (по адресу **http://127.0.0.1:8000/**)

4. Вносим изменения в файл **views.py**

4.2 Добавить контроллер **catalog**

```
def catalog(request):
```

Получаем данные с веб-формы страницы **main.html**

Словарь (который хранит
данные в формате
ключ-значение)

```
    context = {
```

```
        "product": request.POST.get("getproduct")
```

id веб-формы

Ключ

Значение

1. В ключ **product** будут записаны данные, которые пользователь ввел в веб-форму **getproduct**.
2. Ключ **product** можно указать в **catalog.html**, чтобы вывести его значение.

```
    return TemplateResponse(request, "catalog.html", context)
```

5. Создайте директорию **templates** (в рамках Вашего проекта)

В этой директории будут располагаться файлы ***main.html*** и ***catalog.html***.

6. Создайте директорию **static** (в рамках Вашего проекта)

6.1 Создайте директорию **css** (в директории **static**)

В этой директории будут располагаться файлы CSS.

6.2 Создайте директорию **photos** (в директории **static**)

В этой директории будут располагаться графические файлы (например, фотографии).

Аналогичным образом можно создавать директории для различных типов файлов, которые будут использоваться в проекте (файлы в формате SVG, PNG, JS и пр.)

7. Создайте файл **main.html** (в директории **templates**)

```
{% load static %}
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <title>Главная страница</title>
    <link rel="stylesheet" href="{% static 'css/main.css' %}" />
  </head>
  <body>
    <div><a href="{% url 'catalog' %}">Каталог</a></div>
    <div>
      <form method="post" action="catalog/">
        {% csrf_token %}
        <label for="getproduct">Введите название продукта:</label>
        <input type="text" id="getproduct" name="getproduct" />
        <input type="submit" value="Отправить" />
      </form>
    </div>
  </body>
</html>
```

Подключение CSS

В атрибуте **action** указывается *маршрут*, который обслуживает веб-форму (принимает значения веб-форм)

Создание гиперссылки

Имя маршрута (см. файл **urls.py**)

Защита веб-формы от CSRF-атак

id веб-формы (см. файл **views.py**)

Текст (который будет введен пользователем в данную веб-форму) будет отправлен на сервер и записан в ключ **product** (см. файл **views.py**).

8. Создайте файл **catalog.html** (в директории **templates**)

```
{% load static %}
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <title>Каталог</title>
    <link rel="stylesheet" href="{% static 'css/catalog.css' %}" />
  </head>
<body>

  <div><a href="{% url 'main' %}">Главная страница</a></div>

  <div>{{ product }}</div>

</body>
</html>
```

Шаблонизатор Django, во время сборки страницы, автоматически заменит директиву с ключом **{{ product }}** на значение, которое оно содержит.

Ключ **product** (см. контроллер **catalog** в файле **views.py**)

Обратите внимание, что для передачи значений переменных и ключей используются двойные фигурные скобки.